Objektorientierte Programmierung

WOCHE 1: REFRESHER & CLEAN CODE

PETER BERNHARDT

März 2026

1

Teil 1: Einleitung & Die Relevanz von Code-Qualität

2

Willkommen zum Vertiefungsfach

In diesem Semester bewegen wir uns weg von der rein funktionalen Programmierung ("Es läuft") hin zum Software-Engineering.

Wir betrachten Software als ein langlebiges Produkt, das über Jahre oder Jahrzehnte gewartet und erweitert werden muss.

Lernziele für heute:

- Den Übergang vom skriptbasierten zum objektorientierten Denken meistern.
- Die Prinzipien von Clean Code verstehen und anwenden können.
- Code-Gerüche (Code Smells) identifizieren und durch Refactoring beseitigen.

3

Warum "Clean Code" in der Verwaltungsinformatik?

In der öffentlichen Verwaltung begegnen uns oft Altsysteme (Legacy Code), die schwer zu ändern sind.

Schlechter Code führt zu:

- **Technischen Schulden:** Jede schnelle, unsaubere Lösung kostet uns später doppelt so viel Zeit.
- **Fehleranfälligkeit:** Unübersichtlicher Code versteckt Bugs effektiver.
- **Wartungskosten:** In der IT entfallen oft bis zu 80% der Gesamtkosten auf die Wartung nach dem Release.

Clean Code ist kein Selbstzweck, sondern eine ökonomische Notwendigkeit für nachhaltige Digitalisierung.

4

Das “Broken Window” Prinzip

Dieses **Konzept aus der Kriminologie** lässt sich perfekt auf die Softwareentwicklung übertragen:

> *Ein ungepflegtes Gebäude mit einem kaputten Fenster signalisiert: “Hier kümmert sich niemand”. Bald folgen weitere Fenster und Vandalismus.*

Übertragung auf Code: Wenn ein Modul bereits unordentlich ist, neigen Entwickler dazu, ihren neuen Code ebenfalls unsauber hinzuzufügen (“Auf das bisschen Chaos kommt es auch nicht mehr an”).

Gegenmaßnahme: Halten Sie den Code von Anfang an sauber. Reparieren Sie “kaputte Fenster” sofort.

5

Teil 2: Der Mindset-Shift zur OOP

6

Prozedurales vs. Objektorientiertes Denken

Viele Studierende kommen aus der Welt der Skripte (Python, C, JavaScript-Snippets). Dort ist der Code eine lineare Abfolge von Befehlen.

Das prozedurale Paradigma:

- Fokus auf den Algorithmus: “Was muss nacheinander passieren?”
- Daten (Variablen) und Funktionen (Logik) sind getrennt.

Das objektorientierte Paradigma:

- Fokus auf die Domäne: “Welche Akteure (Objekte) gibt es und was ist ihre Verantwortung?”
- Daten und die Logik, die diese manipuliert, bilden eine Einheit (Kapselung).

7

Beispiel: Der prozedurale Ansatz (Anti-Pattern)

Oft sieht man Code, bei dem Daten lose als Arrays oder einfache Variablen verwaltet werden. Das führt zu unübersichtlichen Methodenaufrufen:

```
// Daten sind lose
string vorname = "Max";
string nachname = "Mustermann";
int alter = 25;

// Die Logik schwebt frei im Raum
void DruckeBuerger(string v, string n, int a) {
    Console.WriteLine($"{v} {n}, {a} Jahre");
}
```

Problem: Wenn wir ein neues Datum (z.B. Adresse) hinzufügen, müssen wir alle Methodenköpfe im gesamten Programm ändern.

8

Beispiel: Der objektorientierte Ansatz (Ziel)

Wir bündeln Daten und Logik dort, wo sie hingehören: In das Objekt selbst.

```
public class Buerger {
    public string Vorname { get; set; }
    public string Nachname { get; set; }
    public int Alter { get; set; }

    public void Drucken() {
        Console.WriteLine($"{Vorname} {Nachname}, {Alter} Jahre");
    }
}

// Nutzung
var person = new Buerger { Vorname="Max", Nachname="Mustermann", Alter=25 };
person.Drucken();
```

Das Objekt verwaltet seinen eigenen Zustand und bietet Dienstleistungen (Methoden) an.

9

Teil 3: C# Refresher – Die Bausteine

10

Klassen und Properties

Properties sind in C# das Standardwerkzeug für den Datenzugriff. Vermeiden Sie öffentliche Felder (Fields)!

```
public class Antrag {
    // Auto-Property: Compiler erstellt das Feld im Hintergrund
    public DateTime Eingangsdatum { get; set; }

    // Property mit Logik (Validation)
    private string _steuerId;
    public string SteuerId {
        get => _steuerId;
        set {
            if (value.Length != 11) throw new
                                    ArgumentException("Ungültig");
            _steuerId = value;
        }
    }
}
```

Properties ermöglichen es uns, später Logik hinzuzufügen, ohne die Schnittstelle der Klasse zu brechen.

11

Kapselung und Sichtbarkeiten

Das Geheimnisprinzip (Information Hiding) besagt, dass ein Objekt seine internen Details verbergen sollte.

`private`: Standard für alle Felder. Nur die Klasse selbst kennt diese Daten.

`public`: Die öffentliche Schnittstelle der Klasse (der "Vertrag" mit der Außenwelt).

`protected`: Erlaubt Erben (Subklassen), auf Details zuzugreifen, hält sie aber vor dem Rest der Welt verborgen.

`internal`: Sichtbar innerhalb des gleichen Projekts/Assemblies.

Regel: So restriktiv wie möglich, so offen wie nötig.

12

Die Rolle von Konstruktoren

Ein Objekt sollte zu jedem Zeitpunkt in einem gültigen Zustand sein. Ein Konstruktor erzwingt die Bereitstellung von Pflichtdaten.

```
public class Akte {
    public string Aktenzeichen { get; }

    // Konstruktor erzwingt das Aktenzeichen
    public Akte(string az) {
        if (string.IsNullOrWhiteSpace(az))
            throw new ArgumentException("Az darf
nicht leer sein.");
        Aktenzeichen = az;
    }
}
```

Ohne Konstruktor könnte jemand eine `Akte` ohne `Aktenzeichen` erstellen – ein logischer Fehler in unserem System.

13

Teil 4: Clean Code – Die Kunst der Namensgebung

14

Namen als Dokumentation

Guter Code benötigt kaum Kommentare, weil die Namen der Variablen, Methoden und Klassen die Absicht des Autors erklären.

Die goldene Regel: Wählen Sie Namen, die das “Warum” und “Was” erklären, nicht das “Wie”.

Schlecht: `List<string> sList = new List<string>();` (Sagt nur etwas über den Typ aus).

Gut: `List<string> angemeldeteTeilnehmer = new List<string>();` (Sagt etwas über den fachlichen Inhalt aus).

15

Keine Angst vor langen Namen

In der modernen Softwareentwicklung (mit Auto-Completion) sind kurze, kryptische Namen ein Hindernis.

Vermeiden: `int d;`, `var ret = true;`, `void Proc(int i);`

Bevorzugen: `int tageSeitAntragseingang;`, `var istWahlberechtigt = true;`, `void RegistriereBuerger(int buergerId);`

Besonders in der Verwaltung sind Fachbegriffe (z.B. `Verwaltungsverfahrensgesetz`) oft lang. Schreiben Sie diese aus, statt eigene, unklare Abkürzungen zu erfinden.

16

Namenskonventionen in C

Halten Sie sich an den Microsoft-Standard für Konsistenz:

PascalCase: Für Klassen, Methoden und Properties (`BuergerService`, `BerechneBetrag`, `Name`).

camelCase: Für lokale Variablen und Parameter (`aktuellerAntrag`, `id`).

_camelCase (mit Unterstrich): Für private Felder (`_datenbankVerbindung`).

Einheitlichkeit reduziert die kognitive Last beim Lesen von fremdem Code enorm.

17

Teil 5: Funktionen und Methoden-Design

18

Die “Do One Thing” Regel (SRP)

Eine Methode sollte genau eine Aufgabe erfüllen und diese vollständig erledigen. Wenn Sie eine Methode mit “Und” beschreiben müssen, ist sie wahrscheinlich zu groß.

Schlecht: `void ValidierenUndSpeichern(Antrag a)`

Besser: Trennen in `bool IstAntragValide(Antrag a)` und `void SpeichereAntrag(Antrag a)`.

Kleine Methoden sind leichter zu verstehen, einfacher zu testen und fördern die Wiederverwendbarkeit.

19

Die optimale Länge einer Methode

Es gibt keine harte Grenze, aber eine gute Faustregel ist:

- Eine Methode sollte idealerweise auf einen Bildschirm passen, ohne dass man scrollen muss (ca. 5 – 20 Zeilen).
- Wenn eine Methode 50 oder 100 Zeilen hat, enthält sie fast immer mehrere Verantwortlichkeiten, die extrahiert werden sollten (Refactoring: Extract Method).

20

Abstraktionsebenen in Methoden

Mischen Sie nicht verschiedene logische Ebenen in einer Funktion.

High-Level: “Prüfe, ob der Bürger förderfähig ist.”

Low-Level: “Suche in der SQL-Datenbank nach der ID 42.”

Eine gute Methode liest sich wie ein Inhaltsverzeichnis: Sie ruft andere spezialisierte Methoden auf, statt selbst alle Details zu implementieren.

21

Die Anzahl der Parameter

Je mehr Parameter eine Methode hat, desto schwerer ist sie zu verstehen und zu testen.

0-2 Parameter: Ideal.

3 Parameter: Akzeptabel, wenn nötig.

4+ Parameter: Warnsignal! Fassen Sie zusammengehörige Parameter in einer eigenen Klasse oder Struktur zusammen (Parameter Object).

22

Teil 6: Code Smells und Anti-Patterns

23

Magic Numbers und Magic Strings

Zahlen oder Texte, die ohne Erklärung im Code stehen, nennt man “magisch”. Sie sind gefährlich, weil ihre Bedeutung unklar ist.

Schlecht: `if (antrag.Status == 4) { ... }` (Was ist 4?)

Lösung: Nutzen Sie `Enums`.

Besser: `if (antrag.Status == Status.Abgelehnt) { ... }`

Enums bieten Typsicherheit und machen den Code lesbar wie einen englischen (oder deutschen) Satz.

24

DRY – Don’t Repeat Yourself

Wiederholen Sie sich nicht. Redundanter Code ist die größte Fehlerquelle bei späteren Änderungen.

Wenn Sie die gleiche Logik (z.B. die Berechnung der Mehrwertsteuer) an drei Stellen haben, werden Sie bei einer Gesetzesänderung garantiert eine Stelle vergessen zu aktualisieren. **Lösung:** Logik in eine zentrale Methode oder Klasse auslagern.

25

KISS – Keep It Simple, Stupid

Wählen Sie immer die einfachste Lösung, die das Problem löst. Vermeiden Sie “Over-Engineering”.

Schreiben Sie keinen hochkomplexen, generischen Algorithmus, wenn eine einfache Schleife ausreicht. “Schlauer” Code ist oft schwer zu warten. Ihr Ziel ist **klarer** Code, kein beeindruckend komplizierter Code.

26

Teil 7: Kommentare und Dokumentation

27

Der Mythos vom “selbstdokumentierenden Code”

Code kann nie das “Warum” einer geschäftlichen Entscheidung erklären, aber er sollte immer das “Was” erklären können.

Vermeiden: Kommentare, die den Code nur wiederholen.
`i++; // i um eins erhöhen`

Vermeiden: Kommentare, die schlechten Code rechtfertigen. `// Dieser Teil ist etwas unübersichtlich, aber er funktioniert...`-> **Lösung: Refactoring!**

28

Wann sind Kommentare sinnvoll?

Nutzen Sie Kommentare sparsam und gezielt für:

1. **Rechtliche Hinweise:** Lizenzen, Copyright.
2. **Warnungen:**
   `// Achtung: Der Aufruf dauert ca. 30 Sekunden (API-Limit).`
3. **Absichtserklärung (Das Warum):**
   `// Wir nutzen hier den Algorithmus X, um Kompatibilität mit System Y zu gewährleisten.`
4. **TODOs:** Um geplante Aufgaben temporär zu markieren.

29

Teil 8: Refactoring in der Praxis

30

Was ist Refactoring genau?

Refactoring bedeutet: Die interne Struktur des Codes verbessern, **ohne** das beobachtbare äußere Verhalten zu ändern.

Es ist wie das Aufräumen einer Küche während des Kochens: Es dauert kurzzeitig länger, verhindert aber, dass man später im Chaos versinkt und die Bestellung (das Release) verpasst.

31

Die Boy Scout Rule (Pfadfinder-Regel)

“Hinterlasse den Code immer ein Stück sauberer, als du ihn vorgefunden hast.”

Wenn jeder Entwickler bei jeder Änderung nur eine kleine Variable besser benennt oder eine zu lange Methode aufteilt, verbessert sich die Code-Qualität kontinuierlich, statt langsam zu verrotten.

32

Teil 9: Zusammenfassung

33

Die 5 wichtigsten Takeaways für heute

1. Code wird für Menschen geschrieben, nicht nur für den Compiler.
2. Namen sind Ihre wichtigste Dokumentation.
3. Methoden sollten klein sein und nur eine Aufgabe haben.
4. Vermeiden Sie Redundanz (DRY) und magische Werte (Enums nutzen).
5. Refactoring ist ein integraler Bestandteil der täglichen Arbeit.

34